

Doom Maze Prototype - Project Report

Authors: Louis De Gruyter, Sebastiaan Delodder **Date:** 2026-04-28

Course: Computer Graphics

Project summary

This project is a small first-person maze experience built with Three.js. It combines procedural maze generation, a 3D world built from the same maze data, a minimap with shortest-path guidance, and collision-aware movement. The focus of the work was on implementing and understanding core computer graphics and game-logic techniques rather than on asset production.

System overview (high level)

The runtime flow is: generate one logical maze -> derive a minimap and 3D geometry from it -> build a collision octree from wall tiles -> start a first-person controller and per-frame loop. All gameplay systems share the same maze data, which keeps the minimap, rendering, and collision consistent.

Team contributions

Please replace the placeholders with the actual names and tasks. TODO: Fill in specific contributions for each team member.

- **Louis De Gruyter:** [Tasks owned, e.g., maze generation, pathfinding, minimap]
- **Sebastiaan Delodder:** [Tasks owned, e.g., 3D world building, materials, lighting, collision]
- **Joint work:** [Tasks done together, e.g., integration, debugging, tuning]

Random maze generation

The maze is generated as a grid of cells that starts fully walled. The generator then applies several passes so the output looks like a mix of rooms and corridors instead of a pure corridor labyrinth.

Key choices and reasoning:

- **Odd-sized grid and carved corridors:** The grid is forced to odd dimensions so corridor carving can jump by two cells and leave a one-cell wall between lanes. This gives clean maze structure and avoids thin walls.
- **Room-first pass:** The generator attempts to place small rooms first, then larger rooms. This guarantees some open areas before the corridor carve begins.
- **Randomized depth-first carving:** Corridors are carved with a stack-based randomized depth-first search, giving a classic maze feel while still allowing themed floor runs.
- **Region IDs + connector walls:** Rooms and corridor trees are tagged with region IDs. A later pass opens walls between regions to create loops and prevent isolated islands.
- **Dead-end softening:** Some dead ends are extended or expanded into small rooms to reduce frustration and add variation.
- **Teleport placement:** Teleport pads are placed as paired links using a scoring function (open area preference, distance from start/goal, distance between pads, and cross-region bonus). If a disconnected

component remains, one teleport pair is added to link it back to the reachable area.

- **Wall materials after layout:** Wall materials are assigned only after floor themes are known, so walls inherit the dominant adjacent floor theme.

What we learned:

- A single DFS carve is fast but produces monotonous mazes. Adding room and connector passes creates a more readable space without sacrificing randomness.
- Teleports are useful both as a gameplay mechanic and as a safety net to guarantee connectivity.

Shortest path finding

The minimap route uses breadth-first search (BFS) over the maze grid. Each floor cell is a node; edges connect the four cardinal neighbors. Teleport pads are modeled as additional edges so the path can include teleport hops when they shorten the route.

Why BFS:

- All walk steps have uniform cost, so BFS returns the true shortest path in number of steps.
- The maze sizes are small enough that BFS is very fast and simpler than A* or Dijkstra.

Runtime behavior:

- The path is recomputed when the player enters a new cell or clicks a new destination.
- The drawn route is simplified for open areas with line-of-sight checks so the overlay looks clean.

3D world organization (geometric primitives)

The 3D world is built directly from the maze grid. Each cell becomes a small set of geometric primitives:

- **Floor and ceiling tiles:** Plane primitives per walkable cell.
- **Walls:** Box primitives for wall cells, sized to tile dimensions and wall height.
- **Teleport markers:** Spheres slightly above the floor.
- **Decorations:** Small clusters of simple primitives (cylinders, boxes, cones, torus, octahedrons, dodecahedrons) chosen per region theme.

All meshes are grouped into a single maze root group. This keeps updates and rebuilds simple when the user changes the maze theme.

Moving in 3D and visibility

Movement model

- **Input and camera:** Mouse movement drives yaw and pitch, with pointer lock for first-person control. Keyboard input defines local movement, then rotates into world space by yaw.
- **Predictive collision:** Movement is tested before committing. The player is represented as an upright box around the camera eye position. Each axis (X, Z, then Y) is resolved independently, producing natural wall sliding.
- **Jump and gravity:** Jump adds vertical velocity, gravity integrates each frame, and the camera height is clamped to the grounded eye level when landing.
- **Delta clamping:** The per-frame delta is clamped to avoid large jumps after tab switching or lag.

Frustum culling and hidden surface determination

- **Frustum culling:** No custom culling is implemented. The engine relies on Three.js default object-level frustum culling, which skips rendering meshes outside the camera frustum.
- **Hidden surface removal:** The GPU depth buffer (z-buffer) handles visibility between overlapping surfaces. Back-face culling is used by default for most materials; ceilings are rendered double-sided where needed.

Lighting choices

The lighting is a layered setup that balances readability with atmosphere:

- **Hemisphere light:** Provides a soft sky/ground gradient so surfaces never become fully black.
- **Ambient light:** Adds a mild global lift to reduce harsh contrast.
- **Local point lights:** Region decorations and special tiles spawn point lights with theme-colored tints. A small subset casts shadows for depth and visual focus.
- **Emissive materials:** Teleport pads and special markers add emissive highlights that read well in darker corridors.

The renderer uses physically correct lighting, soft shadow maps, and ACES tone mapping to keep exposure consistent across themes.

Texture mapping strategies

Textures are generated procedurally and mapped in a world-aligned way:

- **Procedural surface textures:** Albedo and bump maps are painted in code using patterns like brick, stone, panel, rock, and crystal. This avoids external asset dependencies and keeps themes consistent.
- **World-aligned UVs:** UVs are computed from world-space positions instead of per-tile local UVs. This makes neighboring tiles line up seamlessly and prevents visible seams at tile borders.
- **Tile-aware repetition:** Repeat factors are chosen per material family (walls vs floors) so the scale reads correctly in the maze.
- **Special surfaces:** Start, goal, and teleport pads use emissive details and distinct patterns for legibility.

Octree build time and optimization ideas

Measured build time: Ran on laptop with an Intel i5-13420H and 24GB DDR5 RAM. Times are averages over 100 runs per maze size.

Benchmark results:

(index)	size	runs	mean_ms	mean_us	std_ms	min_ms	max_ms
0	10	100	0.000000	0	0.000000	0.000000	0.000000
1	100	100	0.060000	60	0.237487	0.000000	1.000000
2	1000	100	0.600000	600	0.616441	0.000000	2.000000
3	10000	100	5.870000	5870	1.689112	4.000000	14.000000
4	100000	100	76.110000	76110	7.356487	64.000000	105.000000

Notes:

- The octree is built once per maze creation from wall collision boxes.
- Build cost scales with the number of wall tiles and the chosen depth/thresholds.

Ideas for further speedups:

- Merge adjacent wall tiles into larger boxes to reduce entry count.
- Use a loose octree so items that touch split planes can still be pushed deeper.
- Build the tree in a Web Worker to avoid blocking the main thread during rebuilds.
- Switch to a fixed uniform grid for very regular, grid-aligned mazes.
- Cache and reuse octree nodes when only textures/themes change.

Collision detection with the octree

Collision is handled in two phases:

1. **Broad phase:** The player bounding box queries the octree to retrieve only nearby wall boxes.
2. **Narrow phase:** Exact AABB intersection tests are performed against those candidates.

The axis-by-axis resolution (X then Z then Y) prevents sticking and produces smooth wall sliding. This approach is simple, stable, and fast for tile-based worlds.

Ray tracing using the octree (optional)

Status: Not implemented in the current project.

Planned approach:

- Traverse the octree by intersecting the ray with node bounds.
- Visit children in order of entry distance to allow early exit on first hit.

Expected complexity:

- Average: $O(\log n + k)$, where k is the number of candidate primitives tested.
- Worst case: $O(n)$ if the ray intersects many nodes or the tree becomes unbalanced.

Octree vs kd-tree and BSP-tree

Why an octree was a good fit here

- The maze is axis-aligned and grid-based, which matches an octree's axis-aligned subdivision.
- Insertion is simple and supports rebuilds when the maze is regenerated.
- Querying with an AABB (player box) is straightforward and fast in practice.

When I would definitely use an octree

- Axis-aligned, relatively uniform 3D spaces (grid mazes, voxel worlds, blocky environments).
- When I need fast broad-phase queries with simple implementation and rebuilds.

Limitations compared to kd-tree and BSP-tree

- Octrees subdivide space uniformly, which can waste nodes in uneven distributions.

- Items that cross split planes stay higher in the tree, reducing pruning effectiveness.
- kd-trees can adapt splits to data distribution, often improving query efficiency for non-uniform scenes.
- BSP trees can split along polygon planes and are excellent for static indoor scenes with complex occlusion and visibility calculations.

When I would avoid an octree

- Highly non-uniform scenes with long, thin geometry where adaptive splits matter.
- Static indoor levels where BSP-style visibility or portal culling is a priority.
- Large point clouds or irregular geometry sets where a kd-tree provides better balance.

Challenges and lessons learned

- **Predictive collision vs push-out correction:** Pushing the player out of walls caused unstable corrections when multiple walls overlapped. Predictive collision (rejecting invalid moves) was more stable and easier to reason about.
- **UV alignment:** Tile-local UVs made seams obvious. World-aligned UVs fixed this and made the maze feel continuous.
- **Connectivity:** Random room placement can isolate regions. Teleport-based reconnect logic ensured the maze stayed fully traversable.

Image placeholders

Replace these with final renders and screenshots.

 Figure 1: Overall maze view  Figure 2: First-person corridor view
 Figure 3: Room with decorations and local lights  Figure 4: Minimap with route overlay
 Figure 5: Teleport pads in-world  Figure 6: Theme comparison grid
 Figure 7: Collision debug visualization  Figure 8: Material close-ups